BRAINWARE UNIVERSITY

ML PROJECT-BASED LEARNING REPORT (WEEK 2)

1. Basic Information

Title of the Project:	**Electricity Bill Calculator & ATM Transaction Simulator (Program 1 & 2)**
Name(s) of Student(s):	**Soumyadwip Das (BWU/BTS/25/169)** **Ankita Paul (BWU/BTS/25/180)** **Sonia Naskar (BWU/BTS/25/195)** **Joyeta Mondal (BWU/BTS/25/214)** **Sayantan Bharati (BWU/BTS/25/503)**
Programme & Semester:	**B.Tech CSE & 3rd Semester**
Course Name & Code:	**Python Programming Lab (BES09013)**
Name of the Guide/Supervisor:	**MR. PRANAB GHARAI & MR. INDRANIL DAS**

2. Introduction and Background

Week 2 of the laboratory work focused on applying core Python programming concepts - conditional logic, functions, in-memory data modelling, and menu-driven control flow - to two independent console-based applications: an Electricity Bill Calculator (Program 1) and an ATM Transaction Simulator (Program 2).

Program 1 explores slab-wise (tiered) tariff billing, conditional discounts, and fixed service charges through a terminal-based electricity billing tool. Program 2 builds on the account-holder and record-handling concepts introduced in Program 1, extending them into stateful account records, credential verification, and transaction validation for a simulated ATM. Both programs were implemented using core Python data structures and control flow only, without relying on external libraries or a persistent database, and both present their interface through a consistent bordered, banner-styled command-line design.

3. Objectives of the Project

3a. Objectives - Program 1: Electricity Bill Calculator

- Implement a menu-driven terminal application for electricity bill calculation
- Develop slab-wise (tiered) billing logic based on units consumed
- Apply a conditional discount rule for bills exceeding a defined threshold
- Incorporate a fixed service charge into the final bill amount
- Explore basic in-memory data storage using Python dictionaries to track multiple users

- Validate user input handling using exception handling techniques

3b. Objectives - Program 2: ATM Transaction Simulator

- Design and implement a console-based ATM simulator with a clear, boxed menu interface
- Develop an authentication routine that validates a card number and PIN before granting access
- Implement balance enquiry, cash withdrawal, and cash deposit operations with input validation
- Develop a PIN-change feature that verifies the existing PIN before allowing an update
- Explore in-memory data modelling of bank accounts using Python dictionaries
- Test the simulator against valid and invalid inputs to observe program behaviour

4. Problem Statement / Driving Question

4a. Program 1: Electricity Bill Calculator

How can a terminal-based application accurately compute an electricity bill using slab-wise tariff rates, apply an applicable discount, add a fixed charge, and manage multiple user records within a simple menu-driven interface?

4b. Program 2: ATM Transaction Simulator

How can everyday ATM operations - authentication, balance enquiry, withdrawal, deposit, and PIN management - be accurately simulated in a console environment using Python, while ensuring that invalid credentials, insufficient balances, and incorrect PINs are handled safely?

5. Methodology

5a. Methodology - Program 1: Electricity Bill Calculator

1. Program Design

- Designed a menu-driven interface offering four options: Calculate Bill, Display Bill Status, Show All User Info, and Exit
- Implemented reusable box-drawing helper functions (box_top, box_bottom, box_divider, box_line, print_boxed) to render a bordered terminal UI, along with an ASCII-art banner

2. Billing Logic Implementation

- Developed the calc(unit) function implementing four consumption slabs (0-100, 101-200, 201-300, and above 300 units) with progressively increasing per-unit rates
- Applied a 10% discount rule that is triggered when the computed bill amount exceeds Rs. 1000
- Added a fixed charge of Rs. 50 to every computed bill after the discount is applied

3. Data Handling & Validation

- Implemented savedata() to record each user's consumed units and final bill in an in-memory dictionary (ALL_DATA)
- Validated numeric input for unit consumption using try/except around the float conversion, displaying an error message on invalid entries
- Tested the 'Display Bill Status' and 'Show All User Info' menu options to confirm that stored data is retrieved and displayed correctly

5b. Methodology - Program 2: ATM Transaction Simulator

1. Program Structure

- Developed reusable box-drawing helper functions (box_top, box_bottom, box_divider, box_line, print_boxed) to render a consistent bordered menu and message interface
- Displayed an ASCII-art banner and centred title on program start-up for a polished terminal experience

2. Account Data Modelling

- Represented bank accounts using a Python dictionary (ACCOUNTS) keyed by card number, storing the holder's name, PIN, and balance
- Explored the trade-offs of in-memory storage versus a persistent database for future enhancement

3. Core Transaction Logic

- Implemented an authenticate() function that validates the card number and PIN pair before any transaction proceeds
- Developed withdraw() and deposit() functions that update the account balance, with withdrawal validated against available funds
- Implemented change_pin() to update the stored PIN only after verifying the current PIN

4. Menu-Driven Control Flow

- Built a continuous while-loop menu offering Check Balance, Withdraw Cash, Deposit Cash, Change PIN, and Exit options
- Validated numeric inputs for withdrawal and deposit amounts, rejecting non-numeric or non-positive values
- Tested the simulator with correct and incorrect card/PIN combinations, insufficient-balance withdrawals, and PIN changes to confirm correct error handling

6. Expected Outcomes / Deliverables

6a. Program 1: Electricity Bill Calculator

- A working terminal-based electricity bill calculator with slab-wise billing logic
- Verified discount and fixed-charge computation across a range of unit values
- A menu system that supports repeated bill calculations for multiple users within a single session
- Basic in-memory record-keeping of each user's computed bill

- Documented groundwork for future enhancements, such as persistent (file-based) storage or a graphical interface

6b. Program 2: ATM Transaction Simulator

- A working console-based ATM Transaction Simulator supporting authentication, balance enquiry, withdrawal, deposit, and PIN change
- Input validation for transaction amounts and PIN/card credentials
- A clear, boxed terminal interface for menus and transaction feedback
- Documented understanding of in-memory account modelling using dictionaries
- Identified enhancement opportunities such as PIN-attempt lockout and persistent storage for Week 3

7. Core Logic

7a. Core Logic - Program 1: Electricity Bill Calculator

The calc() function is the core of the program. It determines the bill using slab-wise rates, applies a threshold-based discount, and adds a fixed charge:

```
def calc(unit):
    bill = 0
    discount = 0
    charge = FIXED_CHARGE

    # Slab-wise tariff: rate increases as consumption rises
    if unit <= 100:
        bill = unit * 1.5
    elif unit <= 200:
        bill = (100 * 1.5) + ((unit - 100) * 2.5)
    elif unit <= 300:
        bill = (100 * 1.5) + (100 * 2.5) + ((unit - 200) * 4.0)
    else:
        bill = (100 * 1.5) + (100 * 2.5) + (100 * 4.0) + ((unit - 300) * 6.0)

    # 10% discount applies only above a Rs. 1000 threshold
    if bill > 1000:
        discount = DISCOUNT_RATE

    amount_after_discount = bill - (bill * discount) / 100
    final_bill = amount_after_discount + charge   # fixed charge added last

    return final_bill
```

In short: the function accumulates cost slab by slab as units increase, applies a 10% discount only when the pre-charge bill exceeds Rs. 1000, and finally adds the fixed Rs. 50 service charge before returning the total.

7b. Core Logic - Program 2: ATM Transaction Simulator

The authentication and withdrawal routines form the operational core of the simulator. authenticate() gatekeeps every transaction by verifying the card number and PIN before any account data is accessed, while withdraw() enforces that a transaction cannot exceed the available balance.

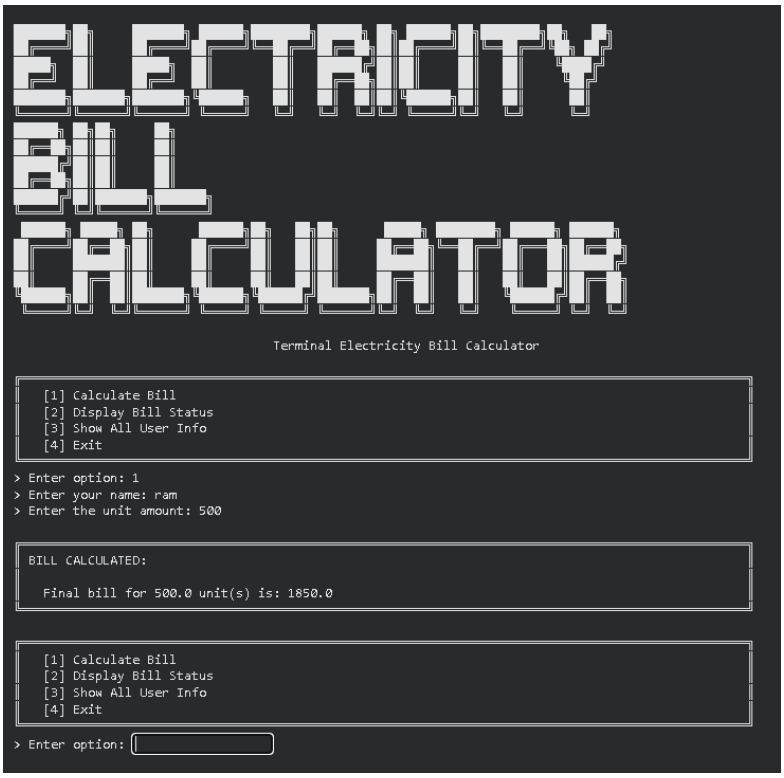
```
def authenticate(card_no, pin):
    if card_no in ACCOUNTS and ACCOUNTS[card_no]["pin"] == pin:
        return True                # credentials match
    return False                   # invalid card/PIN

def withdraw(card_no, amount):
    if amount <= ACCOUNTS[card_no]["balance"]:
        ACCOUNTS[card_no]["balance"] -= amount   # deduct funds
        return True
    return False                   # insufficient balance
```

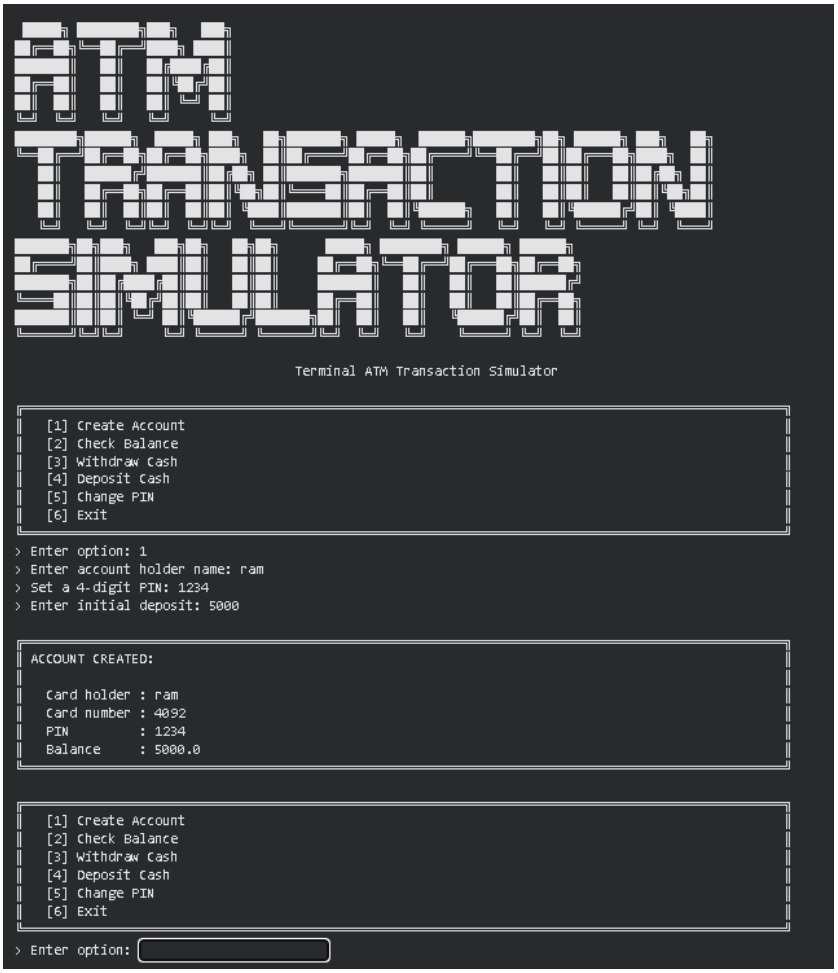
Note: MAX_ATTEMPTS is currently declared but not yet enforced in the login flow; wiring it into authenticate() to lock a card after repeated failed attempts is planned as a future enhancement.

8. Output Screenshot

8a. Program 1: Electricity Bill Calculator



8b. Program 2: ATM Transaction Simulator



9. Project Timeline

Week	Date	Activity	Status
Week 1	10/07/2026	Student Grade Management System	**Completed**
Week 2	**17/07/2026**	**Program 1: Electricity Bill Calculator**	**Current Week**
Week 2	**17/07/2026**	**Program 2: ATM Transaction Simulator**	**Current Week**
Week 3	24/07/2026	Program 1: Bank Management System	**Upcoming**
Week 3	24/07/2026	Program 2: Hotel Booking System	**Upcoming**
Week 3	24/07/2026	Program 3: Library Management System	**Upcoming**
Week 4	31/07/2026	Program 1: Password Strength Checker	**Upcoming**
Week 4	31/07/2026	Program 2: Email Validation System	**Upcoming**
Week 5	TBD	Text Encryption & Decryption Tool	**Upcoming**

10. Tools & Technologies Used

Tool / Technology	Purpose
Python 3.x	Core programming language used to implement both the Electricity Bill Calculator and the ATM Transaction Simulator
Python Standard Library / Built-in input() & print()	Core language features used across both programs: functions, dictionaries, f-strings, exception handling, and command-line interaction
Python Dictionaries	In-memory data structure used to manage user billing records (Electricity Bill Calculator) and account records (ATM Simulator)
String Formatting (f-strings)	Formatting billing, balance, withdrawal and deposit details for display
Functions & Control Flow	Modularising billing logic, authentication, withdrawal, deposit and PIN-change logic

Tool / Technology	Purpose
Command-Line / Terminal Interface	Execution environment for running and testing both menu-driven programs
Text Editor / IDE	Used to write and debug both Python source files

11. References

- Python Official Documentation: https://docs.python.org/3/
- Python Exception Handling (try/except) Guide: https://docs.python.org/3/tutorial/errors.html
- Python f-strings & String Formatting: https://docs.python.org/3/tutorial/inputoutput.html
- Python string formatting (f-strings) - PEP 498: https://peps.python.org/pep-0498/
- Python Dictionaries - Data Structures Guide: https://docs.python.org/3/tutorial/datastructures.html
- GeeksforGeeks - Electricity Bill Calculator (Python) tutorials
- Brainware University - Machine Learning Course Guidelines

12. Signatures

Signature of Student(s): **Date of Submission:**

Signature of Guide/Supervisor: